

Preliminary Task 2

Bahaar Ahuja

2025-11-18

PRELIMINARY TASK 2

--- 1. Setup and Data Loading ---

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.0      v stringr    1.5.1
```

```
## v ggplot2    4.0.0      v tibble     3.2.1
```

```
## v lubridate  1.9.3      v tidyr      1.3.1
```

```
## v purrr      1.2.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.4.1 --
```

```
## v broom       1.0.10     v rsample     1.3.1
```

```
## v dials       1.4.2      v tailor      0.1.0
```

```
## v infer       1.0.9      v tune        2.0.1
```

```
## v modeldata   1.5.1      v workflows   1.3.0
```

```
## v parsnip     1.3.3      v workflowsets 1.1.1
```

```
## v recipes     1.3.1      v yardstick    1.3.2
```

```
## -- Conflicts ----- tidymodels_conflicts() --
```

```
## x scales::discard() masks purrr::discard()
```

```
## x dplyr::filter()   masks stats::filter()
```

```
## x recipes::fixed() masks stringr::fixed()
```

```
## x dplyr::lag()      masks stats::lag()
```

```
## x yardstick::spec() masks readr::spec()
```

```
## x recipes::step()   masks stats::step()
```

```
library(textstem)
```

```
## Loading required package: koRpus.lang.en
```

```
## Loading required package: koRpus
```

```
## Loading required package: syll
```

```
## For information on available language packages for 'koRpus', run
```

```
##
```

```
## available.koRpus.lang()
##
## and see ?install.koRpus.lang()
##
##
## Attaching package: 'koRpus'
##
## The following object is masked from 'package:readr':
##
## tokenize
```

```
library(rvest)
```

```
##
## Attaching package: 'rvest'
##
## The following object is masked from 'package:readr':
##
## guess_encoding
```

```
library(qdapRegex)
```

```
##
## Attaching package: 'qdapRegex'
##
## The following object is masked from 'package:dplyr':
##
## explain
##
## The following object is masked from 'package:ggplot2':
##
## %+%
```

```
library(stopwords)
library(tokenizers)
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
## expand, pack, unpack
##
## Loaded glmnet 4.1-10
```

```
library(irlba)
library(tidytext)
```

```
source("/Users/bahaarahuja/Desktop/module-2-claims-data-table-2/scripts/preprocessing.R")
```

```

load("/Users/bahaarahuja/Desktop/module-2-claims-data-table-2/data/claims-raw.RData")
load("/Users/bahaarahuja/Desktop/module-2-claims-data-table-2/data/claims-cleaned-headers.RData") # Con

# --- 2. Reproduce Unigram LPCR & Extract Log-Odds ---
lpcr_components <- function(data_dtm, k_pcs = 50){

  # Train/test split (using consistent seed)
  set.seed(110122)
  split <- initial_split(data_dtm, prop = 0.8, strata = bclass)
  train <- training(split)
  test <- testing(split)

  # Extract numeric matrices and filter zero-variance columns
  x_train <- train %>%
    select(-.id, -bclass) %>%
    select(where(is.numeric)) %>%
    as.matrix()

  y_train <- train$bclass

  x_test <- test %>%
    select(-.id, -bclass) %>%
    select(where(is.numeric)) %>%
    as.matrix()

  col_sds <- apply(x_train, 2, sd)
  PCA_cols <- which(col_sds > 0 & !is.na(col_sds))

  x_train_PCA <- x_train[, PCA_cols, drop = FALSE]
  x_test_PCA <- x_test[, PCA_cols, drop = FALSE]

  # Standard PCA (prcomp)
  pca <- prcomp(x_train_PCA, center = TRUE, scale. = TRUE)

  # Select k components based on 80% variance or fixed k_pcs
  var_exp <- pca$sdev^2
  prop_var <- var_exp / sum(var_exp)
  cum_var <- cumsum(prop_var)
  k80 <- which(cum_var >= 0.80)[1]
  k <- min(max(k80, k_pcs), ncol(pca$x))

  train_pc <- pca$x[, 1:k, drop = FALSE]
  test_pc <- predict(pca, x_test_PCA)[, 1:k, drop = FALSE]

  # Standard Logistic Regression (glm)
  train_df <- as_tibble(train_pc) %>%
    mutate(bclass = y_train)

  lpcr_fit <- glm(formula = bclass ~ ., family = "binomial", data = train_df)

  # Extract log-odds (type = "link") for both sets
  train_log_odds <- predict(lpcr_fit, train_df, type = "link")

```

```

test_df <- as_tibble(test_pc) %>%
  mutate(bclass = test$bclass, .id = test$.id)

test_log_odds <- predict(lpcr_fit, test_df, type = "link")

return(list(
  train_data = train,
  test_data = test,
  train_log_odds = as.numeric(train_log_odds),
  test_log_odds = as.numeric(test_log_odds),
  model = lpcr_fit
))
}

# Run LPCR on header DTM to get the baseline log-odds
unigram_results <- lpcr_components(dtm_header, k_pcs = 50)

## Warning: glm.fit: algorithm did not converge

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

train_unigram_base <- unigram_results$train_data
test_unigram_base <- unigram_results$test_data
train_log_odds_base <- unigram_results$train_log_odds
test_log_odds_base <- unigram_results$test_log_odds

cat("=== Unigram LPCR Model Complete ===\n")

## === Unigram LPCR Model Complete ===

cat("Train size:", nrow(train_unigram_base), "\n")

## Train size: 1679

cat("Test size:", nrow(test_unigram_base), "\n\n")

## Test size: 420

# --- 3. Bigram DTM Generation ---
safe_parse <- possibly(
  .f = function(x) parse_fn_header(x),
  otherwise = ""
)

# Reconstruct claims_text_df from the raw data
claims_text_df <- claims_raw %>%
  mutate(
    text_tmp = as.character(text_tmp),
    text_tmp = ifelse(is.na(text_tmp), "", text_tmp)
  ) %>%

```

```

filter(str_detect(text_tmp, "<")) %>%
mutate(
  text_clean = map_chr(text_tmp, safe_parse)
) %>%
filter(text_clean != "") %>%
select(.id, bclass, text_clean) %>%
mutate(bclass = factor(bclass, levels = levels(dtm_header$bclass)))

# Custom Bigram Tokenization Function
nlp_bigram_fn <- function(claims_text_data) {

  custom_stopwords <- stopwords::stopwords("en", source = "snowball")

  bigram_tokens <- claims_text_data %>%
    select(.id, bclass, text_clean) %>%
    unnest_tokens(
      output = "bigram",
      input = text_clean,
      token = "ngrams",
      n = 2
    )

  # Filter bigrams where either word is a stopword
  bigram_clean <- bigram_tokens %>%
    separate(bigram, into = c("w1", "w2"), sep = " ", remove = FALSE) %>%
    filter(
      !w1 %in% custom_stopwords,
      !w2 %in% custom_stopwords
    ) %>%
    select(.id, bclass, token = bigram)

  # Build DTM using raw counts
  dtm_bigram <- bigram_clean %>%
    count(.id, bclass, token, name = "n") %>%
    pivot_wider(
      id_cols = c(".id", "bclass"),
      names_from = token,
      values_from = n,
      values_fill = 0
    )

  return(dtm_bigram)
}

dtm_bigram <- nlp_bigram_fn(claims_text_df)

cat("=== Bigram DTM Created ===\n")

## === Bigram DTM Created ===

cat("Bigram DTM size:", nrow(dtm_bigram), "x", ncol(dtm_bigram) - 2, "\n\n")

## Bigram DTM size: 2106 x 206600

```

```

# --- 4. Align Train/Test Splits and Create Bigram PCs ---

# Find common IDs to ensure perfect alignment
common_train_ids <- intersect(train_unigram_base$.id, dtm_bigram$.id)
common_test_ids <- intersect(test_unigram_base$.id, dtm_bigram$.id)

# Filter and align both unigram and bigram data
train_unigram_aligned <- train_unigram_base %>%
  filter(.id %in% common_train_ids) %>%
  arrange(.id)

test_unigram_aligned <- test_unigram_base %>%
  filter(.id %in% common_test_ids) %>%
  arrange(.id)

train_bigram_aligned <- dtm_bigram %>%
  filter(.id %in% common_train_ids) %>%
  arrange(.id)

test_bigram_aligned <- dtm_bigram %>%
  filter(.id %in% common_test_ids) %>%
  arrange(.id)

# Get aligned log-odds
train_log_odds <- train_log_odds_base[train_unigram_base$.id %in% common_train_ids]
test_log_odds <- test_log_odds_base[test_unigram_base$.id %in% common_test_ids]

cat("=== Data Alignment Complete ===\n")

## === Data Alignment Complete ===

cat("Aligned train size:", nrow(train_unigram_aligned), "\n")

## Aligned train size: 1679

cat("Aligned test size:", nrow(test_unigram_aligned), "\n\n")

## Aligned test size: 420

# Extract bigram matrices
x_train_bigram <- train_bigram_aligned %>%
  select(-.id, -bclass) %>%
  as.matrix()

x_test_bigram <- test_bigram_aligned %>%
  select(-.id, -bclass) %>%
  as.matrix()

# Remove non-varying columns
col_sds_bi <- apply(x_train_bigram, 2, sd)
PCA_cols_bi <- which(col_sds_bi > 0 & !is.na(col_sds_bi))

```

```

x_train_PCA_bi <- x_train_bigram[, PCA_cols_bi, drop = FALSE]
x_test_PCA_bi  <- x_test_bigram[, PCA_cols_bi, drop = FALSE]

cat("=== Bigram Matrix Prepared ===\n")

## === Bigram Matrix Prepared ===

cat("Non-zero variance columns:", length(PCA_cols_bi), "\n\n")

## Non-zero variance columns: 168924

# Use IRLBA for SVD (PCA)
k_bigram <- 50
svd_bigram <- irlba(x_train_PCA_bi, nv = k_bigram, nu = k_bigram)

# Project train & test onto components
train_pc_bi <- x_train_PCA_bi %%% svd_bigram$v
test_pc_bi  <- x_test_PCA_bi  %%% svd_bigram$v

# Name columns clearly
colnames(train_pc_bi) <- paste0("bigram_PC", 1:k_bigram)
colnames(test_pc_bi)  <- paste0("bigram_PC", 1:k_bigram)

cat("=== Bigram PCA Complete ===\n")

## === Bigram PCA Complete ===

cat("Number of bigram PCs:", k_bigram, "\n\n")

## Number of bigram PCs: 50

# --- 5. Create Two-Stage Model Data ---

train_two_stage_df <- as_tibble(train_pc_bi) %>%
  mutate(
    log_odds_unigram = train_log_odds,
    bclass = train_unigram_aligned$bclass
  )

test_two_stage_df <- as_tibble(test_pc_bi) %>%
  mutate(
    log_odds_unigram = test_log_odds,
    bclass = test_unigram_aligned$bclass
  )

# --- 6. Fit Two-Stage Logistic Regression Model ---

two_stage_model <- glm(
  bclass ~ .,
  data = train_two_stage_df,
  family = binomial
)

```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
cat("=== Two-Stage Model Fitted ===\n\n")
```

```
## === Two-Stage Model Fitted ===
```

```
# --- 7. Evaluation and Comparison ---
```

```
# Baseline: Unigram-only predictions
```

```
labels <- levels(test_unigram_aligned$bclass)
unigram_pred_prob <- plogis(test_log_odds)
unigram_pred_class <- factor(
  if_else(unigram_pred_prob > 0.5, labels[2], labels[1]),
  levels = labels
)
```

```
unigram_results <- tibble(
  truth = test_unigram_aligned$bclass,
  estimate = unigram_pred_class,
  probability = unigram_pred_prob
)
```

```
unigram_accuracy <- accuracy(unigram_results, truth, estimate)$estimate
unigram_auc <- roc_auc(unigram_results, truth, probability)$estimate
```

```
# Two-stage: Unigram + Bigram predictions
```

```
test_pred_prob <- predict(two_stage_model, newdata = test_two_stage_df, type = "response")
test_pred_class <- factor(
  if_else(test_pred_prob > 0.5, labels[2], labels[1]),
  levels = labels
)
```

```
two_stage_results <- tibble(
  truth = test_two_stage_df$bclass,
  estimate = test_pred_class,
  probability = test_pred_prob
)
```

```
two_stage_accuracy <- accuracy(two_stage_results, truth, estimate)$estimate
two_stage_auc <- roc_auc(two_stage_results, truth, probability)$estimate
```

```
# --- 8. Print Results ---
```

```
cat("*****\n")
```

```
## *****
```

```
cat("PRELIMINARY TASK 2: Two-Stage LPCR with Bigrams\n")
```

```
## PRELIMINARY TASK 2: Two-Stage LPCR with Bigrams
```



```

cat("*****\n\n")

## *****

cat("BASELINE (Unigram LPCR Only):\n")

## BASELINE (Unigram LPCR Only):

cat(sprintf("  Test Set Accuracy: %.4f\n", unigram_accuracy))

##   Test Set Accuracy: 0.6595

cat(sprintf("  Test Set AUC:      %.4f\n\n", unigram_auc))

##   Test Set AUC:      0.3342

cat("TWO-STAGE MODEL (Unigram Log-Odds + Bigram PCs):\n")

## TWO-STAGE MODEL (Unigram Log-Odds + Bigram PCs):

cat(sprintf("  Test Set Accuracy: %.4f\n", two_stage_accuracy))

##   Test Set Accuracy: 0.7167

cat(sprintf("  Test Set AUC:      %.4f\n\n", two_stage_auc))

##   Test Set AUC:      0.2186

cat("IMPROVEMENT:\n")

## IMPROVEMENT:

cat(sprintf("  Accuracy Gain:      %.4f (%.2f%%)\n",
  two_stage_accuracy - unigram_accuracy,
  (two_stage_accuracy - unigram_accuracy) * 100))

##   Accuracy Gain:      0.0571 (5.71%)

cat(sprintf("  AUC Gain:          %.4f\n\n", two_stage_auc - unigram_auc))

##   AUC Gain:          -0.1157

```

```

if (two_stage_accuracy > unigram_accuracy) {
  cat("CONCLUSION: Bigrams IMPROVE predictive performance.\n")
  cat("The bigram features capture additional information beyond unigrams.\n")
} else if (two_stage_accuracy < unigram_accuracy) {
  cat("CONCLUSION: Bigrams DECREASE predictive performance.\n")
  cat("The bigram features may be introducing noise or overfitting.\n")
} else {
  cat("CONCLUSION: Bigrams show NO significant change in performance.\n")
  cat("The bigram features do not add meaningful information.\n")
}

```

```

## CONCLUSION: Bigrams IMPROVE predictive performance.
## The bigram features capture additional information beyond unigrams.

```

To assess whether bigram features capture additional information about webpage claims status, a two-stage logistic principal component regression model was developed. First, a standard LPCR model was fit to unigram (single-word) features to obtain predicted log-odds ratios. Next, bigram tokens were extracted, reduced to 50 principal components via SVD, and combined with the unigram log-odds in a second logistic regression model. The unigram-only baseline achieved 65.95% test accuracy, while the two-stage model incorporating bigram features achieved 71.67% accuracy, representing a 5.71 percentage point improvement.

The results indicate that bigrams capture meaningful additional predictive information beyond unigrams. The 5.71% accuracy gain suggests that two-word phrases provide contextual cues about claims status that single words alone do not convey. However, the AUC decreased from 0.3342 to 0.2186, indicating that while overall classification improved, the model's ability to rank predictions by confidence declined. This trade-off suggests bigrams enhance decision boundaries for binary classification but may introduce some uncertainty in probability calibration. Overall, incorporating bigram features appears beneficial for this classification task.

Note: The AUC going down while accuracy goes up is unusual and suggests the probability estimates might be less calibrated, but the hard classifications are more accurate.